

torch.nn.functional.gumbel_softmax#

https://pytorch.org/docs/stable/generated/torch.nn.functional.gumbel_softmax

Description:

Sample from the Gumbel-Softmax distribution (Link 1 Link 2) and optionally discretize. logits (Tensor) – [..., num_features] unnormalized log probabilities tau (float) – non-negative scalar temperature hard (bool) – if True, the returned samples will be discretized as one-hot vectors, but will be differentiated as if it is the soft sample in autograd dim (int) – A dimension along which softmax will be computed. Default: -1.

Function Signature

```
torch.nn.functional.gumbel_softmax(logits, tau=1,  
hard=False, eps=1e-10, dim=-1)[source]#
```

Parameters

Returns

Return type

Examples::

Returns:

Tensor

Code Examples

```
>>> logits = torch.randn(20, 32)
>>> # Sample soft categorical using reparametrization trick:
>>> F.gumbel_softmax(logits, tau=1, hard=False)
>>> # Sample hard categorical using "Straight-through" trick:
>>> F.gumbel_softmax(logits, tau=1, hard=True)
```

Notes & Warnings

NOTE This function is here for legacy reasons, may be removed from `nn.Functional` in the future.

NOTE The main trick for hard is to do `y_hard - y_soft.detach() + y_soft` It achieves two things: - makes the output value exactly one-hot (since we add then subtract `y_soft` value) - makes the gradient equal to `y_soft` gradient (since we strip all other gradients)

Detailed Documentation

Documentation

Sample from the Gumbel-Softmax distribution ([Link 1](#) [Link 2](#)) and optionally discretize.

logits (Tensor) – [..., num_features] unnormalized log probabilities

tau (float) – non-negative scalar temperature

hard (bool) – if True, the returned samples will be discretized as one-hot vectors, but will be differentiated as if it is the soft sample in autograd

dim (int) – A dimension along which softmax will be computed. Default: -1.

Sampled tensor of same shape as logits from the Gumbel-Softmax distribution. If hard=True, the returned samples will be one-hot, otherwise they will be probability distributions that sum to 1 across dim.

This function is here for legacy reasons, may be removed from nn.Functional in the future.

The main trick for hard is to do $y_{\text{hard}} = y_{\text{soft}}.\text{detach}() + y_{\text{soft}}$

It achieves two things: - makes the output value exactly one-hot (since we add then subtract y_{soft} value) - makes the gradient equal to y_{soft} gradient (since we strip all other gradients)